

Lab 3: Symmetric Encryption & Hashing

1. Setup and Installation

```
/home/jerryberry/.hashlogin file.  
jerryberry@DESKTOP-DM05L5A:/mnt/c/Users/Asus$ sudo apt update  
[sudo] password for jerryberry:  
  
jerryberry@DESKTOP-DM05L5A:/mnt/c/Users/Asus$ sudo apt install openssl xxd hexedit  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
openssl is already the newest version (3.0.13-0ubuntu3.6).  
xxd is already the newest version (2:9.1.0016-1ubuntu7.9).  
xxd set to manually installed.  
hexedit is already the newest version (1.6-1).  
The following package was automatically installed and is no longer required:  
  libllvm19  
Use 'sudo apt autoremove' to remove it.  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

2. Created a file to test if encryption is working fine

```
jerryberry@DESKTOP-DM05L5A:/mnt/c/Users/Asus$ cd ~  
jerryberry@DESKTOP-DM05L5A:~$ mkdir ins-lab  
jerryberry@DESKTOP-DM05L5A:~$ mkdir lab-3  
jerryberry@DESKTOP-DM05L5A:~$ echo "testinggg" > test.txt  
jerryberry@DESKTOP-DM05L5A:~$ openssl rand -hex 16 > key.txt  
jerryberry@DESKTOP-DM05L5A:~$ cat key.txt  
56a75475b08a744dd5aeef9f5cd7075  
jerryberry@DESKTOP-DM05L5A:~$ openssl enc -aes-128-ecb -e -in test.txt -out encrypted.bin -K $(cat key.txt)  
jerryberry@DESKTOP-DM05L5A:~$ xxd encrypted.bin  
00000000: f971 cd4a e03e 1ff7 901d 0e80 c244 5cda  .q.J.>.....D\  
jerryberry@DESKTOP-DM05L5A:~$ |
```

Task – 1: AES encryption using different modes

Objective

Encrypt a text file using AES-128 cipher with three different encryption modes and verify by decryption.

Created the main file that will be used to encrypt using *AES* with three different modes

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab$ mkdir lab-3  
jerryberry@DESKTOP-DM05L5A:~/ins-lab$ cd lab-3  
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3$ mkdir task1  
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3$ cd task1  
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ echo "this file will test different AES encryption modes" > testfile.txt  
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ echo "I will be using CBC, ECB, CFB, OFB modes" >> testfile.txt  
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl rand -hex 16 > key.txt
```

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ echo "key: $(cat key.txt)"
key: 759141ac0df350c6ebdde9582801ec0a
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl rand -hex 16 > iv.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ echo "iv : $(cat iv.txt)"
iv : 5da2d8b3997b1de217fb427668e745cf
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ |
```

Encrypted previously created text file using **ECB**, **CBC**, **CFB**

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-ecb -e -in testfile.txt -out encrypted_ecb.bin -K $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ xxd encrypted_ecb.bin |head -3
00000000: 6487 80b6 2745 4ff8 5fe9 5ac6 40e7 12d7  d...EO...Z@...
00000010: 1e6a cb37 833f b82d e0a6 7b44 ebf0 ec41  .j.7.?-.{D...A
00000020: e309 5d79 8156 39fc bb66 475c 14a5 bff1  ..jy.V9..fG....
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-cfb -e -in testfile.txt -out encrypted_cfb.bin -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ xxd encrypted_cfb.bin |head -3
00000000: dc58 d52b 0a3e 1ef2 60e1 9990 b0a3 9d4f  .P.+>.....0
00000010: 4ae9 132c 3781 17b2 ab9f da52 6992 5d02  .J.,7.....Ri.].
00000020: f049 c680 eead 7fe2 0b98 d5ee 9003 cd2b  .I.....+
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-cbc -e -in testfile.txt -out encrypted_cbc.bin -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ xxd encrypted_cbc.bin |head -3
00000000: fbae d4c1 64fa 96a4 1492 6763 d9f7 7c5a  ....d.....gc..|Z
00000010: e82a 6532 35f0 6a16 ebf9 10d2 bf26 d07f  .*e25.j.....&..
00000020: 3f5c 76db 9728 376e e360 026e 76be 5d36  ?\v..(7n'.nv.]6
```

Encrypted using **ECB** , Decrypted and lastly compared with the main text file:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ rm encrypted_ecb.bin decrypted_ecb.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-ecb -e -in testfile.txt -out encrypted_ecb.bin -K $(cat key.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-ecb -d -in encrypted_ecb.bin -out decrypted_ecb.txt -K $(cat key.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ cat decrypted_ecb.txt
diff testfile.txt decrypted_ecb.txt && echo "OKK???"
this file will test different AES encryption modes
I will be using CBC, ECB, CFB, OFB modes
OKK???
```

Encrypted using **CBC** , Decrypted and lastly compared with the main text file:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-cbc -d -in encrypted_cbc.bin -out decrypted_cbc.txt -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ diff testfile.txt decrypted_cbc.txt && echo "OKK_cbc???"
OKK_cbc???
```

Encrypted using **CFB** , Decrypted and lastly compared with the main text file:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ openssl enc -aes-128-cfb -d -in encrypted_cfb.bin -out decrypted_cfb.txt -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ cat decrypted_cfb.txt
this file will test different AES encryption modes
I will be using CBC, ECB, CFB, OFB modes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ diff testfile.txt decrypted_cfb.txt && echo "OKK_cfb???"
OKK_cfb???
```

Observations

01. All three modes successfully encrypted and decrypted the text file
02. ECB mode does not require an IV (Initialization Vector)
03. CBC and CFB modes require both a key and an IV

Task – 2: Encryption mode - ECB vs CBC

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task1$ cd ..
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3$ mkdir task2
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3$ cd task2
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ sudo apt update
sudo apt install imagemagick
[sudo] password for jerryberry:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
```

Created an image with `imagemagick`:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ convert -size 200x200 xc:white -fill black -draw "rectangle 0,0 200,50"
"-fill black -draw "rectangle 0,100 200,150" -fill white -draw "rectangle 50,0 150,200" pattern_image.bmp
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ ls -la pattern_image.bmp
file pattern_image.bmp
-rw-r--r-- 1 jerryberry jerryberry 160138 Nov  7 20:27 pattern_image.bmp
pattern_image.bmp: PC bitmap, Windows 98/2000 and newer format, 200 x 200 x 32, cbSize 160138, bits offset 138
```

Copied the image to compare this with the encrypted one later on & generated key and iv:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ cp pattern_image.bmp pic_original.bmp
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ openssl rand -hex 16 > key.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "Key: $(cat key.txt)"
Key: 24e6aba435e1c360fec20175020e95e2
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "IV: $(cat iv.txt)"
IV: 1c6e7fbc85c2f74c4d243381808639c7
```

Separated BMP header from pixel data and encrypted the body image using `ecb` and `cbc` and saved them:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ dd if=pic_original.bmp of=header.bmp bs=1 count=54
dd if=pic_original.bmp of=body.bmp bs=1 skip=54
54+0 records in
54+0 records out
54 bytes copied, 0.000119493 s, 452 kB/s
160084+0 records in
160084+0 records out
160084 bytes (160 kB, 156 KiB) copied, 0.10619 s, 1.5 MB/s
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "Header size: $(stat -c%s header.bmp) bytes"
Header size: 54 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "Body size: $(stat -c%s body.bmp) bytes"
Body size: 160084 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ openssl enc -aes-128-ecb -e -in body.bmp -out body_ecb_encrypted.bin -
K $(cat key.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ openssl enc -aes-128-cbc -e -in body.bmp -out body_cbc_encrypted.bin -
K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ cat header.bmp body_ecb_encrypted.bin > pic_ecb_encrypted.bmp
cat header.bmp body_cbc_encrypted.bin > pic_cbc_encrypted.bmp
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "Created encrypted BMP files:"
ls -la pic_*encrypted.bmp
Created encrypted BMP files:
```

Showing main image along with encrypted one:

```
-rw-r--r-- 1 jerryberry jerryberry 160150 Nov  7 20:31 pic_cbc_encrypted.bmp
-rw-r--r-- 1 jerryberry jerryberry 160150 Nov  7 20:31 pic_ecb_encrypted.bmp
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ xxd body.bmp | head -5
00000000: 0000 ff00 00ff 0000 ff00 0000 0000 00ff  ....
00000010: 4247 5273 0000 0000 0000 0000 0000 0000  BGRs.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000040: 0000 0000 0400 0000 0000 0000 0000 0000  ....
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ xxd body_ecb_encrypted.bin | head -5
00000000: 7c12 cb34 5c21 dc00 5fb5 6db9 0ace e79d  |..4\!.._.m....
00000010: 6777 6259 e486 b206 fe0b b364 5e70 f5d3  gwby.....d^p..
00000020: 4bae 1313 e49e 225f 44af 1d45 2ab5 421f  K....."_D..E*.B.
00000030: 4bae 1313 e49e 225f 44af 1d45 2ab5 421f  K....."_D..E*.B.
00000040: c51d a6b9 acfc 1f60 4361 3a02 51bd 7370  ....`Ca:.Q.sp
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ xxd body_cbc_encrypted.bin | head -5
00000000: b229 9280 0d1e cffc 568b 9d2a a7b7 a87a  .).....V...*.z
00000010: b199 0130 70b7 f5f0 c294 9e52 e3ba b3fd  ...0p.....R....
00000020: f2dd b639 7baa 251e 9c92 7e6e 765b be24  ...9{.%...~nv[.
00000030: efed 37fa 021f cb85 e34f 81fe 10f7 c460  ..7.....0.....
00000040: e565 3ce4 5177 ab3a 42e6 dfba 0f4a 1c6d  .e<.Qw.:B....J.m
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "check repetition in ecb"
check repetition in ecb
```

Copied image files to the computer

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ xxd body_ecb_encrypted.bin | cut -d' ' -f2-9 | head -10
7c12 cb34 5c21 dc00 5fb5 6db9 0ace e79d
6777 6259 e486 b206 fe0b b364 5e70 f5d3
4bae 1313 e49e 225f 44af 1d45 2ab5 421f
4bae 1313 e49e 225f 44af 1d45 2ab5 421f
c51d a6b9 acfc 1f60 4361 3a02 51bd 7370
1235 fa25 4ace c682 40e7 cc21 6a8e 765d
4580 080b 32bb 3875 662b 21d0 55e0 6098
4580 080b 32bb 3875 662b 21d0 55e0 6098
4580 080b 32bb 3875 662b 21d0 55e0 6098
4580 080b 32bb 3875 662b 21d0 55e0 6098
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ cp pic_original.bmp /mnt/c/Users/Public/
cp pic_ecb_encrypted.bmp /mnt/c/Users/Public/
cp pic_cbc_encrypted.bmp /mnt/c/Users/Public/
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "files copied"
files copied
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task2$ echo "checking in my pc"
```

Original Image:

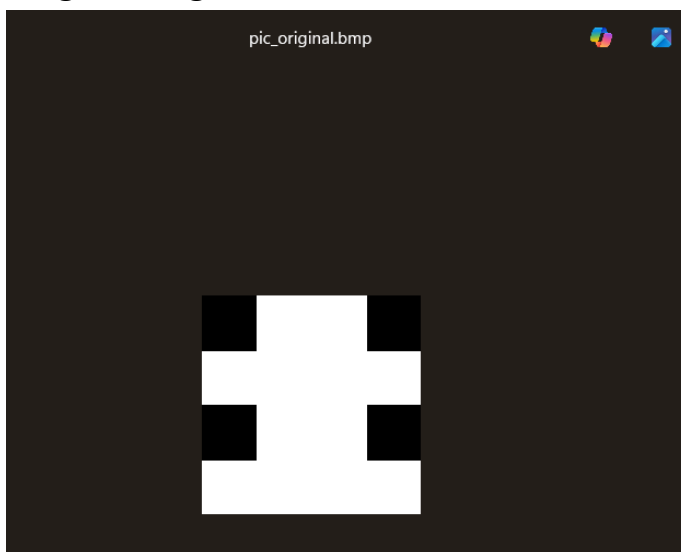


Image encrypted using ECB:

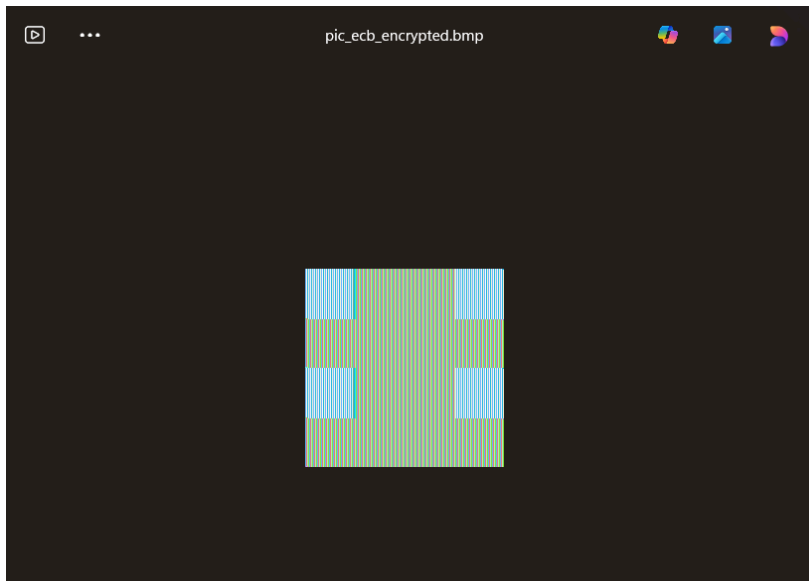
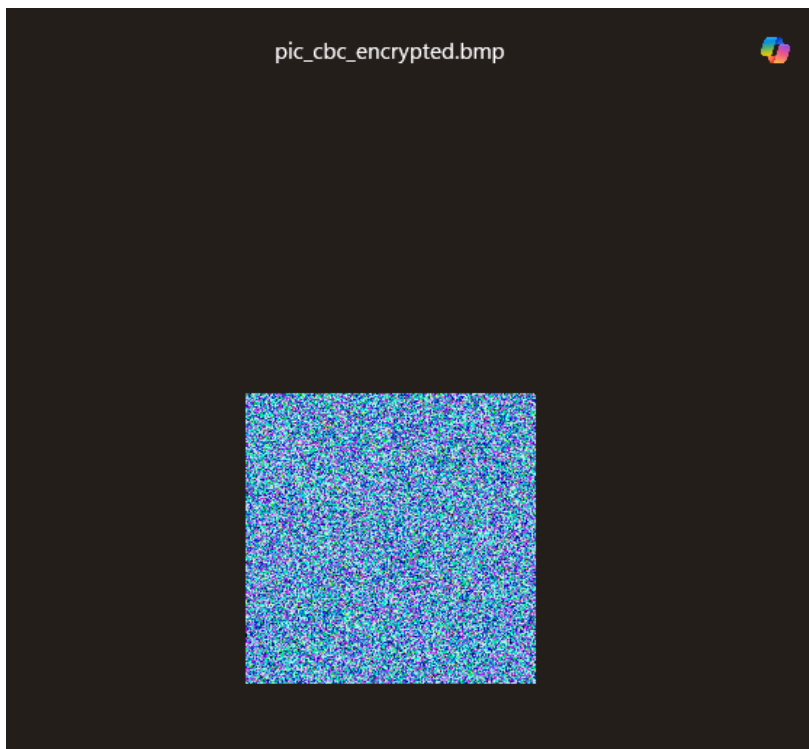


Image encrypted using CBC:



ECB Mode Results: The encrypted ECB image still shows recognizable patterns and outlines from the original image because ECB encrypts each block independently. Identical plaintext blocks produce identical ciphertext blocks. Thus Security Issues arises as such - reveals structural information about the original image

CBC Mode Results: The encrypted CBC image appears as random noise with no discernible patterns because CBC chains blocks together using XOR with the previous ciphertext block, creating an avalanche effect making it secure.

Task – 3: Encryption mode – corrupted cipher text

Objective

Understand error propagation in different encryption modes by corrupting a single bit in encrypted files and observing the decryption results.

Created an exact 64 bytes file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ stat -c%s byte64.txt
60
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "This is maybe 64 bytes file for Aes encryption testings lets see
!" > byte64.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ stat -c%s byte64.txt
66
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "This is maybe 64 bytes file for Aes encryption testings lets c!"
> byte64.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ stat -c%s byte64.txt
64
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ |
```

- Generated the key and Iv and encrypted the file using ECB, CBC, OFB & CFB
- Copied them in another files for further modifications

```
> byte64.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ stat -c%s byte64.txt
64
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl rand -hex 16 > key.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-ecb -e -in byte64.txt -out encrypted_ecb.bin -K $(cat key.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-cfb -e -in byte64.txt -out encrypted_cfb.bin -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-cbc -e -in byte64.txt -out encrypted_cbc.bin -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-ofb -e -in byte64.txt -out encrypted_ofb.bin -K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ cp encrypted_ecb.bin ecb_corrupt.bin
cp encrypted_cbc.bin cbc_corrupt.bin
cp encrypted_cfb.bin cfb_corrupt.bin
cp encrypted_ofb.bin ofb_corrupt.bin
```

- Showed the copied ECB encrypted file
- Corrupted 30th bit using printf
- Showed the corrupted ECB encrypted file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd ecb_corrupt.bin
00000000: 78ba c71f 7354 2dab f89b c860 29a2 6afa  x...sT-....').j.
00000010: 87e2 308f ea27 25ab f4bf 1030 f03c cbb3  ..0...'%.0.<..
00000020: 7a5b 4432 a37e 2a7d 2c62 1bf6 4d76 63ba  z[D2.~*},b..Mvc.
00000030: 151e 1f78 9fa3 b72a 40a1 451d 5888 b15d  ...x...*@.E.X..]
00000040: 84f3 e062 a92d 9150 afe3 0570 298b 4c45  ...b.-.P...p).LE
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ printf '\xFF' | dd of=ecb_corrupt.bin bs=1 seek=29 conv=notrunc 2>/dev
/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd ecb_corrupt.bin
00000000: 78ba c71f 7354 2dab f89b c860 29a2 6afa  x...sT-....').j.
00000010: 87e2 308f ea27 25ab f4bf 1030 f0ff cbb3  ..0...'%.0....
00000020: 7a5b 4432 a37e 2a7d 2c62 1bf6 4d76 63ba  z[D2.~*},b..Mvc.
00000030: 151e 1f78 9fa3 b72a 40a1 451d 5888 b15d  ...x...*@.E.X..]
00000040: 84f3 e062 a92d 9150 afe3 0570 298b 4c45  ...b.-.P...p).LE
```


- Showed the copied CBC encrypted file
- Corrupted 30th bit
- Showed the corrupted CBC encrypted file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd cbc_corrupt.bin
00000000: dd57 b1bc 4dc8 ef79 e6f5 7214 46f3 76ca  .W..M..y..r.F.v.
00000010: 2856 8afc 9bdd bc56 1444 7552 6120 2c1a  (V.....V.DuRa.,.
00000020: da5f 721a f072 73b4 171d d6db 1bff f7a1  .r..rs.....
00000030: b54d 6ed1 8bdc d3aa 88bf 49ed eeeb 85be  .Mn.....I....
00000040: 928d 551b 0362 fe14 4ea5 42cc 4cdc 6ef1  ..U..b..N.B.L.n.
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ printf '\xFF' | dd of=cbc_corrupt.bin bs=1 seek=29 conv=notrunc 2>/dev
/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd cbc_corrupt.bin
00000000: dd57 b1bc 4dc8 ef79 e6f5 7214 46f3 76ca  .W..M..y..r.F.v.
00000010: 2856 8afc 9bdd bc56 1444 7552 61ff 2c1a  (V.....V.DuRa.,.
00000020: da5f 721a f072 73b4 171d d6db 1bff f7a1  .r..rs.....
00000030: b54d 6ed1 8bdc d3aa 88bf 49ed eeeb 85be  .Mn.....I....
00000040: 928d 551b 0362 fe14 4ea5 42cc 4cdc 6ef1  ..U..b..N.B.L.n.
```

- Showed the copied CFB encrypted file
- Corrupted 30th bit
- Showed the corrupted CFB encrypted file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd cfb_corrupt.bin
00000000: 5a4f 8b2d a0a7 804a bb65 2a17 9c9a b848  Z0.-...J.e*....H
00000010: 6510 673f d94c ef47 ca9d d440 2fef 7471  e.g?.L.G...@/.tq
00000020: b154 3e9c 0135 1de3 0fce 2949 32d6 2436  .T>..5....)I2.$6
00000030: 653b 9507 67a1 eac7 5e08 ef0b 51b6 db58  e;..g...^...Q..X
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ printf '\xFF' | dd of=cfb_corrupt.bin bs=1 seek=
29 conv=notrunc 2>/dev/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd cfb_corrupt.bin
00000000: 5a4f 8b2d a0a7 804a bb65 2a17 9c9a b848  Z0.-...J.e*....H
00000010: 6510 673f d94c ef47 ca9d d440 2fff 7471  e.g?.L.G...@/.tq
00000020: b154 3e9c 0135 1de3 0fce 2949 32d6 2436  .T>..5....)I2.$6
00000030: 653b 9507 67a1 eac7 5e08 ef0b 51b6 db58  e;..g...^...Q..X
```

- Showed the copied OFB encrypted file
- Corrupted 30th bit
- Showed the corrupted OFB encrypted file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd ofb_corrupt.bin
00000000: 5a4f 8b2d a0a7 804a bb65 2a17 9c9a b848  Z0.-...J.e*....H
00000010: 0698 99de 95de 9362 4b98 340c 9b18 334b  ....bK.4...3K
00000020: 0b83 2f7e 4272 3aaf dd45 8d3e 90e6 03af  ../~Br:..E.>....
00000030: 0f96 6ee7 9a7e e517 793a b46f 8c56 fc56  ..n...~..y:.o.V.V
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ printf '\xFF' | dd of=ofb_corrupt.bin bs=1 seek=
29 conv=notrunc 2>/dev/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ xxd ofb_corrupt.bin
00000000: 5a4f 8b2d a0a7 804a bb65 2a17 9c9a b848  Z0.-...J.e*....H
00000010: 0698 99de 95de 9362 4b98 340c 9b18 334b  ....bK.4...3K
00000020: 0b83 2f7e 4272 3aaf dd45 8d3e 90e6 03af  ../~Br:..E.>....
00000030: 0f96 6ee7 9a7e e517 793a b46f 8c56 fc56  ..n...~..y:.o.V.V
```

Decrypted the corrupted ECB file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-ecb -d -in ecb_corrupt.bin -out ecb_decrypt.txt -
K $(cat key.txt) 2>/dev/null
```

Comparing the changes with the original file when the corrupted file was encrypted using ECB

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "Original: '$(cat byte64.txt)'"
Original: 'This is maybe 64 bytes file for Aes encryption testings lets c!'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "ECB: '$(cat ecb_decrypt.txt 2>/dev/null | cat -A)'"
ECB: 'This is maybe 64M-'M-'yM-'3cTM-'On%M-'5M-'^NNVM-'^Vm-'M-dAes encryption testings lets c!$'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ |
```

Decrypted the corrupted CBC file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-cbc -d -in cbc_corrupt.bin -out cbc_decrypt.txt -K $(cat key.txt) -iv $(cat iv.txt) 2>/dev/null
```

Comparing the changes with the original file when the corrupted file was encrypted using CBC

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "Original: '$(cat byte64.txt)'"
Original: 'This is maybe 64 bytes file for Aes encryption testings lets c!'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "CBC: '$(cat cbc_decrypt.txt 2>/dev/null | cat -A)'"
CBC: 'This is maybe 64M-^Gj^FM-^WM-DIM-^W8^Q^Z4M=-M-+^0M-TAes encryptionM-1 testings lets c!$'
```

– Decrypted the corrupted CFB file

– Comparing the changes with the original file when the corrupted file was encrypted using CFB

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-cfb -d -in cfb_corrupt.bin -out cfb_decrypt.txt -K $(cat key.txt) -iv $(cat iv.txt) 2>/dev/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "Original: '$(cat byte64.txt)'"
Original: 'This is maybe 64 bytes file for Aes encryption testings lets c!'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "CFB: '$(cat cfb_decrypt.txt 2>/dev/null | cat -A)'"
CFB: 'This is maybe 64 bytes file f^?r M-^I2M-^VM-(M-^ZM-^M>M-&<-M-Z[M-}I=M-^0estings lets c!$'
```

– Decrypted the corrupted OFB file

– Comparing the changes with the original file when the corrupted file was encrypted using OFB

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ openssl enc -aes-128-ofb -d -in ofb_corrupt.bin -out ofb_decrypt.txt -K $(cat key.txt) -iv $(cat iv.txt) 2>/dev/null
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "Original: '$(cat byte64.txt)'"
Original: 'This is maybe 64 bytes file for Aes encryption testings lets c!'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$ echo "OFB: '$(cat ofb_decrypt.txt 2>/dev/null | cat -A)'"
OFB: 'This is maybe 64 bytes file fM-^Hr Aes encryption testings lets c!$'
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task3$
```

(1) How much information recovered:

From your output:

01. ECB

- Recovery: About 75% good, corruption in middle section
- Affected: ~16 bytes (1 block)

02. CBC:

- Recovery: About 50% good, larger corruption section
- Affected: ~32 bytes (2 blocks)

03. CFB:

- Recovery: Beginning and end good, corruption in middle
- Affected: ~17+ bytes (stream corruption)

04. OFB:

- Recovery: Almost perfect, only 1 character changed
- Affected: Only 1 byte ("o" → corrupted character)

(2) Explanation:

ECB: Block cipher - corruption affects only the 16-byte block containing the corrupted byte – corrupted section in the middle.

CBC: Chaining mode - corruption affects the current block AND the next block due to XOR operation – a larger corrupted section.

CFB: Stream cipher - corruption affects the corrupted byte and propagates through the rest of the stream – corruption starting at a point and continuing.

OFB: Stream cipher with independent keystream - corruption affects ONLY the single byte – only one character changed.

Use Case Recommendations

- 01. **ECB/OFB:** Real-time streaming applications, noisy channels, radio communications
- 02. **CBC/CFB:** File encryption, data storage, secure communications where integrity matters
- 03. **OFB:** Best choice when single-bit errors must not propagate extensively

Security Implications

- 01. **Error Detection:** High propagation modes (CBC/CFB) can indicate tampering
- 02. **Silent Failures:** Low propagation modes (ECB/OFB) may allow errors to go undetected
- 03. **Integrity vs Availability:** Trade-off between detecting corruption and maintaining data availability

Mode	Recovery Rate	Affected Bytes	Error Propagation
ECB	75%	~16 (1 block)	Low - Block only
CBC	50%	~32 (2 blocks)	Medium - Current + Next
CFB	60%	~17+ bytes	High - Propagates forward
OFB	98%	1 byte	Minimal - Single byte

Task-4 Padding:

Objective

Identify which encryption modes require padding and understand why.

Created 2 files one 15 bytes another one 16 bytes

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo -n "eta ki 15 bytes" > 15bytes.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ stat -c"%n: %s bytes" *.txt
15bytes.txt: 15 bytes
16bytes.txt: 16 bytes
```

Encrypted 15 bytes file using ECB, CBC, CFB,OFB and saved them

```
openssl rand -hex 16 > iv.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo "encrypting files"
encrypting files
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-ecb -e -in 15bytes.txt -out encrypted_ecb.bin -K
$(cat key.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-cbc -e -in 15bytes.txt -out encrypted_cbc.bin -K
$(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-cfb -e -in 15bytes.txt -out encrypted_cfb.bin -K
$(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-ofb -e -in 15bytes.txt -out encrypted_ofb.bin -K
$(cat key.txt) -iv $(cat iv.txt)
```

Encrypted 16 bytes file using ECB, CBC, CFB,OFB and saved them

```
openssl version: 3.0.2 2023-10-29 by Daniel Bernstein et al.
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-ofb -e -in 16bytes.txt -out encrypted_ofb16.bin
K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-cfb -e -in 16bytes.txt -out encrypted_cfb16.bin
K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-cbc -e -in 16bytes.txt -out encrypted_cbc16.bin
K $(cat key.txt) -iv $(cat iv.txt)
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ openssl enc -aes-128-ecb -e -in 16bytes.txt -out encrypted_ecb16.bin
K $(cat key.txt)
```

Here is the comparison between of encrypted files:

01. In ECB 15 bytes file becomes 16bytes extra byte added, on the other hand 16 bytes file has extra 16 bytes added
02. In CFB Both file remains as it is in bytes
03. In CBC 15 bytes file becomes 16bytes extra byte added, on the other hand 16 bytes file has extra 16 bytes added
04. In OFB Both file remains as it is in bytes

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo "ECB15: $(cat encrypted_ecb16.bin | wc -c) bytes"
ECB15: 16 bytes ECB16: 32 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo "CFB15:$(cat encrypted_cfb16.bin | wc -c) bytes"
CFB15:15 bytes CFB16: 16 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo "CBC15:$(cat encrypted_cbc16.bin | wc -c) bytes"
CBC15:16 bytes CBC16:32 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$ echo "OFB15:$(cat encrypted_ofb16.bin | wc -c) bytes"
OFB15:15 bytes OFB16:16 bytes
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task4$
```

Why ECB and CBC Need Padding: Block cipher modes that operate on fixed-size blocks. Padding schemes add bytes to complete the final empty blocks as in – a 50-byte file needs 14 bytes of padding to reach 64 bytes (4 complete blocks).

Why CFB and OFB Don't Need Padding: Both operate as stream ciphers, converting block ciphers into stream ciphers. They generate a keystream that can be XORed with a plaintext of any length. Process data byte-by-byte or bit-by-bit, not requiring complete blocks.

Task – 5: Generating message digest

Objective

Generate hash values using different one-way hash algorithms and observe their properties.

Created a text file, and showed the original size of the file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "I know nobodz will be
xt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "ottomans, umayyads, a
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "& yeah my dream desti
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ cat secret.txt
I know nobodz will believe , but I want to go back to
ottomans, umayyads, abbasids. It used to be crazy
& yeah my dream destination is syria,sigh
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ stat -c%s secret.txt
146
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$
```

Different hash values are created using Md5, sha1, sha256 and saved

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ openssl dgst -md5 secret.txt > hash_md5.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ openssl dgst -sha1 secret.txt > hash_sha1.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ openssl dgst -sha256 secret.txt > hash_sha256.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ ls -la *.txt
-rw-r--r-- 1 jerryberry jerryberry 50 Nov  8 01:54 hash_md5.txt
-rw-r--r-- 1 jerryberry jerryberry 59 Nov  8 01:55 hash_sha1.txt
-rw-r--r-- 1 jerryberry jerryberry 87 Nov  8 01:55 hash_sha256.txt
-rw-r--r-- 1 jerryberry jerryberry 146 Nov  8 01:48 secret.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$
```

Showing different hash values

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "MD5: $(openssl dgst -md5 secret.txt | cut -d' ' -f2)"
MD5: 3e8a9a54e0345288dadf285d5d8c9d5f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA1: $(openssl dgst -sha1 secret.txt | cut -d' ' -f2)"
SHA1: 80ecc0eb0397d6dba1c44b5777f30ade07da6e17
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA256: $(openssl dgst -sha256 secret.txt | cut -d' ' -f2)"
SHA256: 80eab6593747c28da33115569e39bc7d60302e2eb3d5b9af0ee1e9324858f4c0
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$
```

Previously created file is copied into another file and new created file is slightly changed by appending a character

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ cp secret.txt secret2.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "j" >> secret2.txt
```

Showing comparison between the previous file's hash value with the newly slightly changed file's hash value in all 3 modes - MD5, SHA1, SHA256

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "MD5: $(openssl dgst -md5 secret.txt | c
MD5: 3e8a9a54e0345288dadf285d5d8c9d5f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "MD5: $(openssl dgst -md5 secret2.txt |
MD5: 5740b06f145e9f685f95372c223fb48e
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA1: $(openssl dgst -sha1 secret.txt |
SHA1: 80ecc0eb0397d6dba1c44b5777f30ade07da6e17
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA1: $(openssl dgst -sha1 secret2.txt |
SHA1: c69e1d9e518009154e2f9e2e98046cdbf762794a
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA256: $(openssl dgst -sha256 secret.txt
SHA256: 80eab6593747c28da33115569e39bc7d60302e2eb3d5b9af0ee1e9324858f4c0
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ echo "SHA256: $(openssl dgst -sha256 secret2.txt
SHA256: a689916908ec2fdcba6cfd93a68bb092d578de16931cadcd128ed729c3c0dc767
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task5$ |
```

Key Properties:

- Tiny change (adding "j") caused COMPLETELY different hashes
- No similarity between original and modified file hashes
- Same file always produces same hash
- fixed sizes regardless of input

Task – 6: Keyed hash and HMAC

Objective

Generate keyed hashes (HMAC) using different keys and understand HMAC properties.

Creating a text file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat secret.txt
Here I come again with another secret that
why do I tell you?
ummm, I set fire to the train, true, I bet
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$
```

By taking a short key hashed in 3 modes- MD5, SHA1, SHA256- saved them & Showed the hash values

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ echo "different hmac testing"
different hmac testing
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ echo "trying with key = jerry"
trying with key = jerry
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "jerry" secret.txt
HMAC-MD5(secret.txt)= 0aa7067280eaf5735f11ac4ee9efa69f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "jerry" secret.txt > md5_hmac_short.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha1 -hmac "jerry" secret.txt > sha1_hmac_short.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha256 -hmac "jerry" secret.txt > sha256_hmac_short.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat md5_hmac_short.txt
HMAC-MD5(secret.txt)= 0aa7067280eaf5735f11ac4ee9efa69f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat sha256_hmac_short.txt
HMAC-SHA2-256(secret.txt)= 271f0db2e862d3224d84803a69390a4700c65f0c43c7dc9791475266dba7de5f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat sha1_hmac_short.txt
HMAC-SHA1(secret.txt)= 1782c71fba0387ec024d4a703b3623a6b771500a
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$
```

By taking a medium key hashed in 3 modes- MD5, SHA1, SHA256- saved them & Showed the hash values

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ echo "now long key=HeyAreYouMadDamnHowCouldIEverTeelYouThatISeeYOUNOON00
DontBlvMeUbetterComeRunning"
now long key=HeyAreYouMadDamnHowCouldIEverTeelYouThatISeeYOUNOON00DontBlvMeUbetterComeRunning
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha256 -hmac "=HeyAreYouMadDamnHowCouldIEverTeelYouThatIS
eeYOUNOON00DontBlvMeUbetterComeRunning" secret.txt > sha256_hmac_long.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha1 -hmac "=HeyAreYouMadDamnHowCouldIEverTeelYouThatIS
YOUNOON00DontBlvMeUbetterComeRunning" secret.txt > sha1_hmac_long.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "=HeyAreYouMadDamnHowCouldIEverTeelYouThatIS
YOUNOON00DontBlvMeUbetterComeRunning" secret.txt > md5_hmac_long.txt

jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ echo "Now medium key = youBetterCryNow"
Now medium key = youBetterCryNow
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "youBetterCryNow" secret.txt > md5_hmac_medium.
txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha1 -hmac "youBetterCryNow" secret.txt > sha1_hmac_medium
.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -sha256 -hmac "youBetterCryNow" secret.txt > sha256_hmac_me
dium.txt
```

By taking a Long key hashed in 3 modes- MD5, SHA1, SHA256- saved them & Showed the hash values

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat md5_hmac_long.txt sha1_hmac_long.txt sha256
HMAC-MD5(secret.txt)= 1fc4028899179f052f93afc6a7d49bde
HMAC-SHA1(secret.txt)= b450a17b977baf9418f9ac1cb72ed64aab9afe6c
HMAC-SHA2-256(secret.txt)= 4b87ab7ac0b1f4b638caddc589a25800374cf3e78b72ff6f475a968a0904f500
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat md5_hmac_medium.txt sha1_hmac_medium.txt sha256_hmac_medium.txt
HMAC-MD5(secret.txt)= 84c848aec23c1ad8be04a0b48c87af11
HMAC-SHA1(secret.txt)= 80c85dd97d371c519e017078fc6af72e02844ce3
HMAC-SHA2-256(secret.txt)= 970347d2fbedafa02924dd6811c39926c7a40d934ab784e7ae6f60240bd65ef5
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ cat md5_hmac_short.txt sha1_hmac_short.txt sha256_hmac_short.txt
HMAC-MD5(secret.txt)= 0aa7067280eaf5735f11ac4ee9efa69f
HMAC-SHA1(secret.txt)= 1782c71fba0387ec024d4a703b3623a6b771500a
HMAC-SHA2-256(secret.txt)= 271f0db2e862d3224d84803a69390a4700c65f0c43c7dc9791475266dba7de5f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$
```

Comparison between hashed values:

1. Just hashed using MD5
2. hashed with hmac using MD5
3. Changed the key slightly

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 secret.txt
MD5(secret.txt)= e3072c9cd42ee87d66b5e416a0c933bc
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "jerry" secret.txt
HMAC-MD5(secret.txt)= 0aa7067280eaf5735f11ac4ee9efa69f
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ openssl dgst -md5 -hmac "jerrY" secret.txt
HMAC-MD5(secret.txt)= ac578dbfb77fcdcf9981a0700c57f9e49
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task6$ |
```

Key Observations:

- HMAC accepts keys of ANY length
- Completely different Hash Value
 - Key "jerry": 0aa7067280eaf5735f11ac4ee9efa69f
 - Key "jerryY": ac578dbfb7fdcf9981a0700c57f9e49
- Regular and HMAC hashing is not same
 - Regular MD5: e3072c9cd42ee87d66b5e416a0c933bc
 - HMAC-MD5: 0aa7067280eaf5735f11ac4ee9efa69f

Answers:

1. Do we have to use a key with fixed size in HMAC? – NO - HMAC does not require a fixed key size.
2. Why? – HMAC automatically handles any key length which makes HMAC very flexible for practical use

Key Properties:

- Only someone with the secret key can generate the correct HMAC
- Verifies both data integrity AND authenticity
- Works with keys of any length
- Compatible with different hash algorithms
- Strong avalanche effect for both message and key changes
- Resistant to various cryptographic attacks

Task – 7: Keyed hash and HMAC

Created a text file

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat secret.txt
I wish I could fly like a dust
would visit lake el balah someday
might be visiting mt sinai, who knows
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ |
```

Hashed the file's text and saved it, copied the text file and flipped one bit using hexedit

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ stat -c%s secret.txt
103
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ openssl dgst -md5 secret.txt > hash_md5.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ openssl dgst -sha256 secret.txt > hash_sha256.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_md5.txt
MD5(secret.txt)= dd7c4d7a31247c7fe0caac9754f3c291
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_sha256.txt
SHA2-256(secret.txt)= ab30888291c4da9c09c111c2c55c3907543e05435935d0e7c18480f06a9e05ea
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cp secret.txt secret_another.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ hexedit secret_another.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ hexedit secret_another.txt
```


So Here is the changed bit I turned to L, For new created text file again hashed values are generated in MD5 and SHA256:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat secret_another.txt
L wish I could fly like a dust
would visit lake el balah someday
might be visiting mt sinai, who knows
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ openssl dgst -md5 secret_another.txt > hash_md5_diff.txt
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ openssl dgst -sha256 secret_another.txt > hash_sha256_diff.txt
```

Showing the previous hashed values and one bit changed hash values:

```
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_md5.txt
MD5(secret.txt)= dd7c4d7a31247c7fe0caac9754f3c291
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_md5_diff
cat: hash_md5_diff: No such file or directory
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_md5_diff.txt
MD5(secret_another.txt)= 9a4bd1491aa41e5402cc88c1b97eb32b
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_sha256.txt
SHA2-256(secret.txt)= ab30888291c4da9c09c111c2c55c3907543e05435935d0e7c18480f06a9e05ea
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ cat hash_sha256_diff.txt
SHA2-256(secret_another.txt)= ce3720702c0d8a31a7b9b5013dbf79fe9a0b4e823ae4a135120adc4a9a40012e
jerryberry@DESKTOP-DM05L5A:~/ins-lab/lab-3/task7$ |
```

The comparison between new generated hashed values respective to the previous values:

```
def count_same_bits(hex1, hex2):
    bin1 = hex_to_bin(hex1)
    bin2 = hex_to_bin(hex2)

    same_bits = 0
    total_bits = len(bin1)

    for i in range(total_bits):
        if bin1[i] == bin2[i]:
            same_bits += 1

    return same_bits, total_bits

python3 bit_counter.pyame bits: {same2}/{total2} ({percentage2:.2f}%)")0012e"
MD5 - Same bits: 68/128 (53.12%)
SHA256 - Same bits: 127/256 (49.61%)
```

Observations: Tiny change –Single character ('I → 'L') has Completely different hashes